Inquiry Response Agent Strategy: RAG & "The Agent Trainer"

To ensure high accuracy in responding to guest inquiries (e.g., "Can 3 adults share a queen bed?", "Check-in time?"), we cannot rely on the LLM's generic knowledge. We must "ground" the AI in your specific property data using **Retrieval-Augmented Generation (RAG)**.

1. The Core Architecture: "Knowledge Base as Code"

We will treat your Hotel/Room data as the "Source of Truth" for the AI.

A. Data Structure (Firestore Enhancements)

Existing `hotels` collection is a good start, but we need to structure it for AI consumption.

- **Structured Facts**: Keep `checkInTime`, `checkOutTime`, `wifiPassword` as fields.
- **Unstructured "Context"**: Create a new sub-collection `hotels/{id}/knowledge_base`.
  - **Documents**: `house_rules`, `getting_around`, `amenity_details`.
  - **Content**: "The pool is open from 8am to 8pm. No glass allowed."

B. The "Semantic Search" Layer (Vector Config)

- **Objective**: When a user asks "Is the pool heated?", the system shouldn't just look for the word "pool". It should understand the *concept* of amenities.
- **Tech**: **Vertex AI Vector Search** or simply using **Gemini Context Caching** (easier for < 1M tokens) to pass the *relevant* hotel rules into the prompt window.

2. The Strategy: "Test, Train, Deploy"

Step 1: "The Agent Trainer" Interface (New Dashboard Page)

We will build a new section in your Visual Dashboard: `/knowledge-base`.

Features:

1. **Fact Sheet Editor**: A clean UI to edit the "Golden Facts" (Wifi, Parking, Access Codes).
2. **The Simulator (Playground)**:
   - **Input**: "I'm arriving at 2 AM, is that okay?"
   - **Context Selector**: Select "Hotel CBL" + "Room 101".
   - **Agent Output**: The AI calls the API and generates a response.
   - **Human Feedback**: You click 👍 or 👎. If 👎, you rewrite the answer. *The system saves this pair as a "Training Example" (Few-Shot Prompting).*

Step 2: Historical Ingestion (The "Memory")

- **Action**: Scrape past Airbnb/Booking.com message threads.
- **Process**:
  i. Anonymize guest names.
  ii. Classify into Q&A pairs. (Q: "Is there a hair dryer?" A: "Yes, under the sink.")
  iii. Store these in `hotels/{id}/past_responses`.
- **Usage**: When a new question comes in, the Agent searches this history first: "*Have we answered this before?*"

Step 3: Retrieval-Augmented Generation (RAG) Flow

1. **Inquiry Arrives**: "How far is the beach?"
2. **Retrieval**:
   - Fetch `hotels/CBL` (Metadata).
   - Fetch `hotels/CBL/knowledge_base` (Rules).
   - Search `hotels/CBL/past_responses` (Vector Search) for similar questions.
3. **Synthesis (Vertex AI)**:
   - Context Provided: "Hotel is 5 mins walk from Camps Bay Beach."
   - Prompt: "Answer the guest using ONLY the context provided. Be polite."
4. **Response**: "The beach is just a 5-minute walk away!"

3. Improving Agent Accuracy (Configuration)

To specifically handle edge cases like "3 adults in a queen bed":

1. **Hard Constraint Config**:
   - Add `max_occupancy` and `bed_config` to `rooms` collection.
   - **Rule**: If (Guests > Max_Occupancy) -> Agent MUST Reply: "I'm sorry, this room only accommodates X people."
2. **Response Tone Settings**:
   - Allow per-hotel "Persona" settings (e.g., "Professional & Formal" vs "Surfer Dude/Relaxed").

4. Implementation Plan (Next Steps)

1. **Update Task List**: Add Phase 6 (Inquiry Agent).
2. **Schema Update**: Create `knowledge_base` sub-collection script.
3. **Build UI**: Create `/dashboard/src/app/knowledge-base` page.
4. **Mock Agent**: Create a Python service `services/inquiry_agent` that accepts a question + hotel_id and returns a grounded answer.